

INTERNSHIP PROJECT ASSIGNMENT

AI CAREER GUIDANCE & ADMISSION INTELLIGENCE PLATFORM

AI/ML Internship Project · 3-Month Practical Program

CANDIDATE NAME

Aryan Ajmera

REFERENCE NUMBER

NFGT/HR/INT/2026/160

INTERNSHIP MODE

Remote

INDUSTRY GUIDE

Aryan Sharma

DATE ISSUED

19 June 2026

EDUCATION

B.Sc

COLLEGE

University of Edinburgh

DURATION

19/06/2026 to 20/09/2026

REPORTING MANAGER

Sonia Sharma

NextGen Forge Technologies

ABIL Imperial Commercial Spaces, Baner, Pune, Maharashtra 411069

www.nextgenforgetechnologies.com | hr@nextgenforgetechnologies.com

Ref: NFGT/HR/INT/2026/160

01

INTERNSHIP ROLE OVERVIEW

This Internship Project Assignment has been officially issued by **NextGen Forge Technologies** to **Aryan Ajmera** as part of the AI/ML Internship Program. The internship is a simulation-based, industry-oriented engagement designed to provide hands-on practical exposure equivalent to the real-world responsibilities of a junior AI/ML engineer in a professional technology organisation.

The candidate will work on the **AI Career Guidance & Admission Intelligence Platform** — a complete, end-to-end AI-powered EdTech application that mirrors actual product development workflows in the education technology industry. From Gemini API integration and RAG pipeline construction to FastAPI backend development, Streamlit frontend implementation, and Render deployment, every task is designed to be achievable at **2–3 hours per day, Monday to Friday**, across 12 weeks.

CANDIDATE NAME:

Aryan Ajmera

REFERENCE NUMBER:

NFGT/HR/INT/2026/160

DOMAIN:

AI/ML Engineering

MODE:

Remote

DURATION:

19/06/2026 to 20/09/2026

INDUSTRY GUIDE:

Aryan Sharma

REPORTING TO:

Aryan Sharma

REPORTING MANAGER:

Sonia Sharma

Internship Focus Areas:

- **Generative AI Integration:** Leverage the Gemini API and LangChain to build intelligent, context-aware career guidance responses.
- **RAG Pipeline Development:** Build a Retrieval-Augmented Generation pipeline using ChromaDB to ground AI responses in curated career and admission knowledge.
- **Full-Stack AI Application:** Connect a Streamlit frontend with a FastAPI backend to deliver a complete, deployable EdTech platform.
- **Professional Workflow:** Follow agile-style monthly phases, documentation standards, GitHub version control, and structured weekly deliverables.
- **Cloud Deployment:** Package and deploy the complete platform to Render for public accessibility and real-world usage.

02

PROJECT DESCRIPTION

PROJECT:

AI Career Guidance & Admission Intelligence Platform

Intermediate Level

The **AI Career Guidance & Admission Intelligence Platform** is a comprehensive, end-to-end AI-powered EdTech system designed to help Class 12 students navigate the critical transition from school to higher education and career. The platform uses Generative AI, Retrieval-Augmented Generation (RAG), and structured

career datasets to deliver personalised stream recommendations, degree guidance, admission information, and AI-generated career roadmaps.

The system simulates the type of intelligent student advisory platform deployed by leading EdTech companies across India and globally. It leverages the Gemini API for natural language intelligence, LangChain for AI orchestration, ChromaDB for knowledge retrieval, FastAPI for the backend, and Streamlit for the interactive student-facing interface — all scoped for **2–3 hours/day, Mon–Fri** across 12 weeks.

Platform Capabilities:

- Personalised career stream recommendations based on student interests, academic background, and aptitude inputs.
- AI-powered degree and college recommendations aligned with selected career streams and admission eligibility.
- RAG-based conversational Q&A grounded in a curated career guidance and admission knowledge base.
- Automated generation of personalised career roadmaps covering educational milestones, skill development, and industry pathways.
- Admission intelligence module providing entrance exam timelines, college comparison, and application guidance.
- PDF report generation for students to download and share their personalised career assessment and roadmap.
- Session-persistent conversation history enabling multi-turn advisory interactions without context loss.

03

PROBLEM STATEMENT

Career confusion among Class 12 students is one of the most pervasive and consequential challenges in the Indian education system. With over 10 million students completing secondary education annually, the majority make degree and stream choices without access to structured, personalised career intelligence — leading to high dropout rates, career misalignment, and underutilisation of potential.

Lack of Personalised Career Guidance at Scale

Existing career counselling services are either prohibitively expensive or inaccessible to students outside metro cities. Generic career aptitude tests provide one-size-fits-all recommendations without accounting for individual academic profiles, interests, or regional admission landscapes.

Information Overload & Admission Complexity

Students and parents face an overwhelming volume of fragmented information about entrance exams, college cutoffs, scholarship opportunities, and degree pathways. The absence of a centralised, intelligent system to filter and personalise this information leads to poor decision-making and missed opportunities.

No AI-Powered Roadmap Generation

Traditional counselling platforms provide static lists of career options without dynamic, personalised roadmaps. Students cannot visualise the step-by-step journey from their current academic standing to their desired career outcome.

Unequal Access to Quality Advisory Resources

Quality career advisory resources disproportionately serve urban, English-medium students. Students from Tier-2 and Tier-3 cities, regional schools, and lower-income households lack access to the same calibre of guidance that influences higher education outcomes.

The **AI Career Guidance & Admission Intelligence Platform** addresses these challenges by delivering an accessible, AI-powered advisory system that provides every student with personalised, data-driven career intelligence — regardless of geography, income, or institutional access.

04

PROJECT FEASIBILITY

This project is fully achievable for B.Sc, B.Tech, and MCA students with beginner-to-intermediate AI/ML knowledge. At **2–3 hours per day, Monday to Friday (~10–15 hours/week)**, each weekly milestone is completable within its allocated timeframe across 12 weeks.

TARGET CANDIDATES:

B.Sc, B.Tech, MCA — CS, IT, AI/ML students

DIFFICULTY LEVEL:

Intermediate — Practical Generative AI Stack

WORKING COMMITMENT:

2–3 Hours/Day · Mon–Fri · ~10–15 Hours/Week

TIMELINE:

12 Weekly Milestones across 3 Monthly Phases

Why this project is feasible at 2–3hr/day:

- **API-First AI Approach:** The project uses the Gemini API for generative intelligence — no model training from scratch, no GPU requirements, and no research-grade ML work. API calls are straightforward to implement with the provided SDK.
- **Streamlit Frontend:** Streamlit eliminates the need for React.js or complex frontend development. Python developers can build fully interactive UIs in hours, not days — making it ideal for B.Sc and early-year students.
- **ChromaDB for RAG:** ChromaDB runs locally and requires no cloud infrastructure setup. Building a functional RAG pipeline with LangChain and ChromaDB is achievable in a single focused week.
- **Modular Phase Structure:** 3 monthly phases — Foundation & MVP, AI Intelligence Layer, and Deployment — each with 4 clearly defined weekly milestones that build progressively on each other.
- **No Research-Level Tasks:** Every task focuses on practical implementation — API integration, UI development, data processing, and deployment — not academic research or novel model architectures.
- **SQLite Database:** SQLite requires zero server setup and is fully supported within the Python standard library — eliminating database infrastructure barriers for early-stage developers.

05

TECHNOLOGIES & TOOLS REQUIRED

The following technology stack has been selected to align with industry-standard Generative AI and EdTech development practices while remaining fully accessible to B.Sc and early-year engineering students.

Technology	Category	Purpose
Python 3.10+	Core Language	Primary language for all backend, AI integration, data processing, and Streamlit UI development.
Streamlit	Frontend Framework	Python-native UI framework for building the interactive student-facing career guidance interface without JavaScript.
FastAPI	Backend Framework	High-performance Python web framework for building the career guidance REST API with auto-generated documentation.
Gemini API	Generative AI	Google's Gemini large language model API for generating personalised career recommendations, roadmaps, and advisory responses.
LangChain	AI Orchestration	Framework for chaining AI calls, managing conversation memory, integrating retrieval, and building the RAG pipeline.
ChromaDB	Vector Database	Local vector database for storing and retrieving embedded career knowledge base documents for the RAG pipeline.
SQLite	Database	Lightweight, serverless relational database for persisting student profiles, session data, and conversation history.
Pandas & NumPy	Data Processing	Data manipulation libraries for processing career datasets, student profile data, and analytical computations.
PyPDF	PDF Processing	PDF parsing library for ingesting career guidance documents, admission brochures, and knowledge base PDFs.
ReportLab	PDF Generation	Professional PDF generation library for creating downloadable personalised career assessment and roadmap reports.
Git & GitHub	Version Control	Source code management, feature branching, and weekly submission workflow.
VS Code	IDE / Editor	Primary development environment with Python, FastAPI, and Streamlit extensions.
Render	Deployment	Cloud platform for hosting the FastAPI backend and Streamlit application in production.

06

KEY FEATURES TO IMPLEMENT

The following features constitute the complete functional scope of the AI Career Guidance & Admission Intelligence Platform. All core features must be implemented and tested before final submission.

#	Feature	Description
01	Student Profile System	Structured onboarding form collecting student academic background, stream interests, aptitude self-assessment, geographic preferences, and career aspirations — stored in SQLite for session persistence.

#	Feature	Description
02	Interest & Aptitude Assessment	Interactive multi-question assessment module that evaluates student interests across STEM, Commerce, Humanities, and professional domains to generate a weighted interest profile for recommendation.
03	AI Stream Recommendation Engine	Gemini API-powered recommendation system that analyses the student profile and interest assessment to suggest suitable Class 12 streams (Science/PCM, Science/PCB, Commerce, Humanities) with justifications.
04	Degree & Career Path Recommendations	Structured recommendation of undergraduate degree programmes, entrance examinations, and career trajectories aligned with the student's selected stream and interest profile.
05	RAG-Based Career Guidance Q&A;	LangChain and ChromaDB-powered Retrieval-Augmented Generation pipeline enabling students to ask natural language questions about careers, colleges, and admission processes — with responses grounded in the knowledge base.
06	AI Career Roadmap Generation	Gemini-generated personalised career roadmaps covering Class 11–12 preparation, entrance exam timelines, undergraduate degree milestones, skill development priorities, and industry entry pathways.
07	Admission Intelligence Module	Curated information on major entrance examinations (JEE, NEET, CLAT, CAT, CUET), application timelines, eligibility criteria, and college selection guidance by stream.
08	College Comparison Module	Side-by-side comparison of colleges by stream, ranking, fees, location, and placement outcomes — enabling data-driven college selection decisions.
09	Conversation History & Session Memory	Persistent conversation history within and across sessions using SQLite and LangChain memory, enabling multi-turn advisory interactions with maintained context.
10	PDF Career Report Generation	ReportLab-generated downloadable PDF report containing the student's interest profile, stream recommendation, career roadmap, and personalised guidance summary for sharing with parents and counsellors.
11	Analytics Dashboard	Usage metrics and aggregated student interest analytics displayed on the Streamlit admin interface for platform performance monitoring.
12	Cloud Deployment on Render	Complete platform deployment to Render with environment variable management, production configuration, and a publicly accessible live URL.

07

3-MONTH WEEKLY TASK BREAKDOWN — 12-WEEK SCHEDULE

Structured across 3 monthly phases with 4 weekly milestones each. At **2–3 hours/day, Mon–Fri (~10–15 hours/week)**, every task is achievable within its allocated week. Complexity increases progressively from foundational setup to advanced AI integration and production deployment.

MONTH 1 — FOUNDATION & MVP (Weeks 1–4)

WEEK 1

~ 2 tasks · ~2 days each

Task W1.1: Environment Setup & Project Architecture

Description:	Set up Python virtual environment and install all required dependencies: fastapi, uvicorn, streamlit, google-generativeai, langchain, chromadb, sqlite3, pandas, numpy, pypdf, reportlab, python-dotenv. Initialise GitHub repository with structured folders: frontend/, backend/, data/, knowledge_base/, reports/, notebooks/. Configure VS Code with Python extensions. Obtain and configure Gemini API key via Google AI Studio. Create .env file for secrets management. Document complete setup in README.md.
Deliverables:	GitHub repo, structured folder architecture, requirements.txt, configured .env, Gemini API key verified, README.md

Task W1.2: Streamlit Frontend — Student Onboarding UI

Description:	Build the Streamlit application entry point (app.py). Design and implement the student onboarding interface: a multi-step form collecting name, city, current class, stream preferences (STEM/Commerce/Humanities), subject interests, academic performance range, and career aspiration keywords. Apply Streamlit custom CSS for a clean, professional interface. Validate all form inputs and display a confirmation summary screen upon completion.
Deliverables:	Streamlit app with multi-step onboarding form, input validation, confirmation screen, session state management

WEEK 2

~ 2 tasks · ~2 days each

Task W2.1: FastAPI Backend — Server Setup & Student Profile API

Description:	Initialise the FastAPI application (main.py). Configure project structure: routers/, models/, schemas/, services/, database/. Implement SQLite database connection using Python's sqlite3 module. Design and create the database schema: Student (id, name, city, stream_preference, interests, academic_level, created_at), Session (id, student_id, created_at), and ConversationHistory (id, session_id, role, message, timestamp). Implement POST /api/students (create profile) and GET /api/students/{id} (retrieve profile) endpoints. Test all endpoints using FastAPI's auto-generated Swagger UI.
Deliverables:	Running FastAPI server, SQLite database with 3 tables, 2 student profile API endpoints, Swagger UI verified

Task W2.2: Interest & Aptitude Assessment System

Description:	Build the interest and aptitude assessment module. Design a structured question bank of 20 questions across 4 domains: STEM (analytical/logical), Commerce (quantitative/business), Humanities (language/social), and Creative Arts/Design. Implement scoring logic that weights responses to generate a normalised interest profile vector (e.g., STEM: 0.72, Commerce: 0.45, Humanities: 0.28). Store assessment results in SQLite. Display results as a visual bar chart in Streamlit using st.bar_chart().
Deliverables:	Question bank (20 questions), scoring algorithm, interest profile vector, SQLite storage, Streamlit bar chart visualisation

WEEK 3

~ 2 tasks · ~2 days each

Task W3.1: Gemini API Integration — Stream Recommendation Engine

Description:	Integrate the Google Gemini API using the google-generativeai Python SDK. Design a structured prompt template for stream recommendation: the prompt receives the student's interest profile vector, academic performance level, and stated career aspirations, and instructs Gemini to recommend one of four streams (Science/PCM, Science/PCB, Commerce, Humanities) with a minimum 150-word personalised justification. Implement the POST /api/recommend/stream FastAPI endpoint. Handle API errors, rate limiting, and empty response edge cases. Display recommendations in Streamlit with formatted markdown output.
Deliverables:	Gemini API integration, stream recommendation prompt template, POST /api/recommend/stream endpoint, Streamlit display, error handling

Task W3.2: Degree & Career Path Recommendations

Description:	Extend the recommendation engine to generate degree and career pathway suggestions. Build a structured prompt that takes the recommended stream and student interest profile as input and instructs Gemini to generate: (a) Top 5 undergraduate degree programmes with descriptions, (b) 3 primary career pathways per degree, (c) Key entrance examinations for each degree, and (d) Estimated timeline to career entry. Implement GET /api/recommend/degrees/{student_id} endpoint. Display recommendations in Streamlit as expandable sections using st.expander().
Deliverables:	Degree recommendation prompt, GET /api/recommend/degrees endpoint, Streamlit expandable degree cards, career pathway display

WEEK 4

~ 2 tasks · ~2 days each

Task W4.1: Basic AI Chatbot & MVP Integration

Description:	Build the conversational AI chatbot interface using Streamlit's chat components (st.chat_message, st.chat_input). Integrate the Gemini API in a simple, stateless chat mode where each student message is sent with a career counselling system prompt. The chatbot should answer general career and admission questions in a warm, informative advisory tone. Implement basic conversation display: user messages on the right, AI responses on the left. Connect the chatbot to the FastAPI backend for session tracking.
Deliverables:	Streamlit chat interface, Gemini-powered chatbot, session-tracked conversation display, FastAPI session endpoint

Task W4.2: MVP Integration & End-to-End Flow Testing

Description:	Integrate all Month 1 components into a cohesive end-to-end application flow: Onboarding Form → Interest Assessment → Stream Recommendation → Degree Recommendations → Chatbot. Test the complete flow with 5 different student personas (e.g., STEM-oriented student targeting engineering, Commerce student targeting CA, Humanities student targeting law). Document all integration issues in a bug log. Resolve all critical flow-breaking bugs. Record a 3-minute screen recording demonstrating the MVP.
Deliverables:	End-to-end MVP flow, 5 persona test cases, bug log, resolved critical issues, 3-minute MVP demo video

MONTH 2 — AI INTELLIGENCE LAYER (Weeks 5–8)

WEEK 5

~ 2 tasks · ~2 days each

Task W5.1: Career Knowledge Base — Document Ingestion & ChromaDB Setup

Description:	Curate a career knowledge base of 15–20 PDF and text documents covering: STEM career guides, Commerce career paths, Humanities career options, JEE/NEET/CLAT/CUET exam guides, top Indian colleges by stream, scholarship programmes, and skill development roadmaps. Use PyPDF to extract text from PDF documents. Implement a document chunking pipeline (chunk size: 500 tokens, overlap: 50 tokens). Initialise ChromaDB as the local vector store and ingest all document chunks with metadata (source, stream_category, topic). Verify retrieval with 5 test queries.
Deliverables:	15–20 curated career documents, PyPDF extraction pipeline, ChromaDB vector store with ingested knowledge base, retrieval test results

Task W5.2: RAG Pipeline — LangChain & Gemini Integration

Description:	Build the full Retrieval-Augmented Generation pipeline using LangChain. Implement the retrieval chain: user query → ChromaDB similarity search (top-k=5 chunks) → retrieved context injected into Gemini prompt → grounded response generation. Design the RAG prompt template to instruct Gemini to answer only from the retrieved context and cite the source document. Implement POST /api/chat/rag FastAPI endpoint. Test the RAG pipeline with 10 career guidance queries and compare response quality against the non-RAG chatbot.
Deliverables:	LangChain RAG chain, ChromaDB retrieval integration, RAG prompt template, POST /api/chat/rag endpoint, quality comparison report

WEEK 6

~ 2 tasks · ~2 days each

Task W6.1: Admission Intelligence Knowledge Base

Description:	Extend the ChromaDB knowledge base with admission-specific content: major entrance exam syllabi summaries, college admission cutoff ranges by stream, application process guides, scholarship eligibility criteria, and important academic calendar dates. Tag all admission documents with metadata: exam_type, stream, year. Build a dedicated retrieval filter in the LangChain chain to query only admission-relevant documents when the student's question is classified as an admission query. Implement basic intent classification using simple keyword matching to route queries to career or admission retrieval chains.
Deliverables:	Admission knowledge base (10+ documents), metadata-filtered retrieval, intent-based query routing, admission Q&A; test cases

Task W6.2: Conversation History & LangChain Memory

Description:	Implement persistent conversation history using LangChain's ConversationBufferMemory integrated with the RAG pipeline, enabling multi-turn advisory conversations where the AI remembers previous questions and recommendations within a session. Store all conversation turns (role, message, timestamp) in the SQLite ConversationHistory table via the FastAPI backend. Build a Streamlit conversation history sidebar displaying the current session's chat history with scroll and clear functionality.
Deliverables:	LangChain ConversationBufferMemory integration, SQLite conversation persistence, Streamlit history sidebar, multi-turn context demonstration

WEEK 7

~ 2 tasks · ~2 days each

Task W7.1: AI Career Roadmap Generation

Description:	Build the career roadmap generation feature. Design a comprehensive roadmap prompt that instructs Gemini to generate a structured, timeline-based career roadmap in JSON format covering: Class 11–12 academic preparation, entrance exam preparation timeline, undergraduate degree milestones (Year 1–4), skill development priorities by year, internship and project milestones, and industry entry pathway. Parse the JSON response and render it as an interactive Streamlit timeline using st.columns() and custom CSS. Implement GET /api/roadmap/{student_id} FastAPI endpoint.
Deliverables:	Roadmap generation prompt, JSON response parsing, Streamlit timeline visualisation, GET /api/roadmap endpoint, 5 sample roadmaps generated

Task W7.2: College Comparison Module

Description:	Build the college comparison feature. Create a structured SQLite table (Colleges) populated with data for 30 colleges across streams: name, stream, city, state, ranking, annual fees, entrance exam required, placement average, and notable alumni. Build a Streamlit comparison interface where students select 2–3 colleges from a dropdown and see a side-by-side comparison table. Add a filter panel for stream, state, and fee range. Implement GET /api/colleges endpoint with query parameter filtering.
Deliverables:	SQLite Colleges table (30 entries), college comparison UI, filter panel, GET /api/colleges endpoint with filters

WEEK 8

~ 2 tasks · ~2 days each

Task W8.1: Analytics Dashboard & Platform Metrics

Description:	Build the analytics and insights module. Implement a Streamlit admin dashboard page (accessible via a password-protected route) displaying platform usage metrics: total students onboarded, interest profile distribution (bar chart), top 5 career streams recommended (pie chart), most frequent chatbot query topics (word cloud using matplotlib), and daily active session counts (line chart). Pull all metrics from SQLite via SQL aggregation queries. Implement GET /api/analytics/summary FastAPI endpoint.
Deliverables:	Admin analytics dashboard, 5 metric visualisations, SQLite aggregation queries, GET /api/analytics/summary endpoint

Task W8.2: API Documentation & Code Refactoring

Description:	Generate comprehensive API documentation using FastAPI's auto-generated Swagger UI (/docs) and ReDoc (/redoc). Ensure all endpoints have complete docstrings, request/response schemas, and example payloads. Refactor the entire codebase: remove debug print statements, add docstrings to all functions and classes, enforce consistent naming conventions (snake_case for Python, kebab-case for API routes), reorganise modules for clarity, and submit a clean, reviewed GitHub pull request.
Deliverables:	FastAPI Swagger documentation complete, refactored codebase, clean GitHub PR, updated README with API reference

MONTH 3 — ADVANCED FEATURES & DEPLOYMENT (Weeks 9–12)

WEEK 9

~ 2 tasks · ~2 days each

Task W9.1: PDF Career Report Generation

Description:	Build the personalised PDF career report generation feature using ReportLab. Design a professional 3–4 page report template including: NextGen Forge header/branding, student profile summary, interest profile bar chart, recommended stream and justification, top 3 degree pathways, AI-generated career roadmap (condensed), and personalised next-step action plan. Implement the report generation service that populates the template with the student's actual data retrieved from SQLite. Add a 'Download My Career Report' button in Streamlit that triggers PDF generation and download.
Deliverables:	ReportLab PDF template, dynamic data population, PDF download button in Streamlit, 5 sample reports generated for test personas

Task W9.2: Enhanced RAG — Multi-Source Retrieval & Source Attribution

Description:	Enhance the RAG pipeline with multi-source retrieval and response attribution. Update the LangChain retrieval chain to return source document metadata alongside each response. Display a 'Sources consulted' expandable section below each AI response in Streamlit showing the document titles referenced. Add a confidence indicator (High/Medium based on retrieval score threshold) to each response. Extend the knowledge base with 10 additional niche career documents (emerging tech careers, alternative career paths, international education options).
Deliverables:	Source attribution in Streamlit UI, confidence indicator, extended knowledge base (25+ documents), multi-source retrieval demonstration

WEEK 10

~ 2 tasks · ~2 days each

Task W10.1: Performance Optimisation & Security Hardening

Description:	Optimise platform performance: implement Streamlit session state caching for repeated Gemini API calls (cache student recommendations for 24 hours to avoid redundant API usage), add FastAPI response caching using <code>functools.lru_cache</code> for the colleges and analytics endpoints, and optimise SQLite queries with appropriate indexes on frequently queried columns (<code>student_id</code> , <code>session_id</code> , <code>stream_preference</code>). Implement security best practices: input sanitisation for all student text inputs to prevent prompt injection, environment variable audit (no hardcoded keys), and API rate limiting using <code>slowapi</code> .
Deliverables:	Streamlit session state caching, FastAPI response caching, SQLite indexes, input sanitisation module, <code>slowapi</code> rate limiting

Task W10.2: Deployment — Render Cloud Setup

Description:	Prepare the complete application for production deployment on Render. Create Procfile and <code>render.yaml</code> configuration for both the FastAPI backend service and the Streamlit frontend service. Configure all environment variables in Render's dashboard (<code>GEMINI_API_KEY</code> , <code>DATABASE_URL</code> , <code>CHROMA_PERSIST_DIR</code>). Execute deployment for both services, verify all API endpoints at the live URLs, and test the complete end-to-end student flow on the production environment. Document the deployment architecture.
Deliverables:	Live FastAPI backend URL on Render, live Streamlit frontend URL on Render, deployment architecture document, production flow verified

WEEK 11

~ 2 tasks · ~2 days each

Task W11.1: End-to-End Production Testing & Bug Resolution

Description:	Conduct thorough end-to-end testing on the live Render deployment across all platform features: student onboarding, interest assessment, stream recommendation, degree recommendations, RAG chatbot, career roadmap generation, college comparison, PDF report download, and conversation history. Test with 5 distinct student personas. Perform basic load testing (10–15 concurrent Streamlit sessions) to assess stability. Document all bugs discovered in a structured bug log. Resolve all critical and high-priority issues. Re-test after resolution.
Deliverables:	Production test report (all features), 5 persona test cases, load test results, resolved critical/high bug log, re-test verification

Task W11.2: Final Documentation & Project Portfolio

Description:	Prepare complete project documentation: comprehensive <code>README.md</code> (project overview, tech stack, setup guide, feature list, API reference summary, deployment links, screenshots of all major views), technical architecture document (system component diagram, RAG pipeline diagram, database schema), and a project report (3–4 pages) covering problem context, solution design, AI integration decisions, platform performance, and key learnings. Package the complete GitHub repository with clean commit history.
Deliverables:	Complete <code>README</code> , architecture diagrams, project report (3–4 pages), clean GitHub repository, all screenshots included

WEEK 12

~ 2 tasks · ~2 days each

Task W12.1: Final Presentation Preparation

Description:	Prepare a professional presentation deck (12–15 slides) covering: problem statement and market context, solution architecture overview, live platform demo walkthrough (all major features), AI/RAG pipeline explanation, Gemini API integration highlights, deployment architecture, platform performance metrics, challenges encountered and solutions applied, and key learnings. Record a 5-minute demo video demonstrating the complete student journey from onboarding to PDF report download. Rehearse the presentation.
Deliverables:	Presentation deck (PPT/PDF, 12–15 slides), 5-minute demo video recording, rehearsal completed

Task W12.2: Final Submission & Presentation Delivery

Description:	Submit the complete project package: GitHub repository link (all code, notebooks, documents), live Render deployment URLs (FastAPI + Streamlit), all documentation (README, architecture diagrams, project report), and presentation deck. Deliver the final presentation to Industry Guide Akash Jha and Reporting Manager Sonia Sharma. Answer Q&A from the panel. Submit all materials via official email to hr@nextgenforgetechnologies.com .
Deliverables:	Final presentation delivered, complete project package submitted via email, Q&A completed, all live URLs verified

08

CANDIDATE RESPONSIBILITIES

As a NextGen Forge Technologies AI/ML intern, Aryan Ajmera is expected to maintain the professional standards and responsibilities outlined below throughout the 3-month engagement.

Optimised & Maintainable Code	Write clean, well-structured, documented Python code following PEP-8 standards. Prioritise readability, modularity, and maintainability across all backend, frontend, and AI integration modules.
Meeting Deadlines	Complete all weekly milestones within the stipulated timeline. Communicate blockers or delays to Industry Guide Akash Jha at least 24 hours in advance with a written explanation.
Testing & Debugging	Independently test all features — Gemini API integration, RAG pipeline, FastAPI endpoints, and Streamlit UI flows — before each weekly submission. Maintain a structured bug log documenting all issues and resolutions.
Documentation Standards	Keep code documented with inline comments and docstrings. Submit weekly progress reports every Friday by 6:00 PM IST to hr@nextgenforgetechnologies.com .
Progress Reviews	Participate actively in all bi-weekly reviews. Come prepared with a progress summary, live demo of completed features, and a clear articulation of next steps and any open blockers.

Professional Communication	Communicate respectfully with Industry Guide Akash Jha and Reporting Manager Sonia Sharma via official channels. Respond to all queries within 24 working hours.
AI Research & Responsible Implementation	Proactively research relevant Gemini API capabilities, LangChain features, and RAG optimisation techniques. Apply findings responsibly — document all AI integration decisions, prompt design rationale, and observed model behaviour.
Professional Conduct	Maintain confidentiality of all project materials including API keys, student data, and platform logic. Adhere to responsible AI practices and represent NextGen Forge Technologies with integrity throughout the engagement.

09

MENTORSHIP & REPORTING STRUCTURE

NextGen Forge Technologies provides a structured mentorship framework to support interns with technical guidance, AI integration expertise, performance feedback, and professional development throughout the 3-month internship.

INDUSTRY GUIDE: Aryan Sharma — AI/ML Engineering Lead	REPORTING MANAGER: Sonia Sharma — HR & Internship Coordinator
REVIEW CADENCE: Bi-Weekly Performance Review (Every 14 Days)	COMMUNICATION CHANNEL: Email / WhatsApp Business (official channel)
QUERY RESPONSE SLA: 24 Hours (working days); immediate for critical blockers	WEEKLY REPORT DUE: Every Friday, 6:00 PM IST — hr@nextgenforgetechnologies.com

Review Schedule:

Review	Week	Agenda	Output
Week 2 Check-in	Week 2	Onboarding, environment setup, Streamlit UI, FastAPI backend progress	Onboarding sign-off, early feedback
Month 1 Review	Week 4	MVP demo — Gemini integration, recommendation engine, chatbot, end-to-end flow	M1 evaluation, Month 2 readiness
Month 2 Mid-Review	Week 6	RAG pipeline, admission knowledge base, conversation history review	Feature quality feedback, adjustments
Month 2 Review	Week 8	Roadmap generation, college comparison, analytics, API documentation	M2 evaluation score, Month 3 plan
Month 3 Mid-Review	Week 10	PDF report generation, enhanced RAG, deployment progress, optimisation	Deployment approval, production checklist
Final Evaluation	Week 12	Final presentation, live platform demo, portfolio review, complete system test	Final scorecard, certificate eligibility

10

EVALUATION CRITERIA

Performance will be evaluated by Aryan Sharma and reviewed by Sonia Sharma at the conclusion of the internship. Bi-weekly review scores contribute cumulatively to the final evaluation.

Parameter	Weight	Assessment Criteria
Technical Implementation	20%	Correctness, completeness, and functionality of all implemented features across 3 months.
AI & RAG Integration Quality	18%	Effectiveness of Gemini API integration, RAG pipeline accuracy, LangChain orchestration, and overall AI response quality.
Task Completion & Timeliness	15%	Percentage of weekly milestones completed on schedule and to the required quality standard.
Code Quality	14%	Readability, structure, docstrings, naming conventions, and adherence to Python and API best practices.
Creativity & Innovation	12%	Novel prompt engineering approaches, UX improvements, self-initiated features, or value-added implementations beyond the scope.
Communication & Reporting	11%	Quality of weekly progress reports, responsiveness, presentation quality, and clarity of documentation.
Professionalism & Consistency	10%	Reliability, conduct, deadline adherence, responsible AI practices, and professional attitude throughout.
TOTAL	100%	Holistic evaluation across all technical, AI/RAG, and professional dimensions.

11

EXPECTED LEARNING OUTCOMES

Upon successful completion of this internship, Aryan Ajmera will have developed the following practical competencies applicable to professional AI/ML engineering and EdTech product development roles:

01	Generative AI Application Development	End-to-end proficiency in building production-ready AI applications using the Gemini API — from API integration and prompt design to response parsing and error handling.
02	RAG Pipeline Engineering	Practical expertise in designing and implementing Retrieval-Augmented Generation pipelines using LangChain and ChromaDB — including document ingestion, chunking, embedding, retrieval, and grounded response generation.
03	LangChain Orchestration	Hands-on experience with LangChain chains, memory modules, retrieval chains, and prompt templates for building stateful, multi-turn AI applications.
04	Full-Stack AI Application Development	Ability to architect and connect a Streamlit frontend with a FastAPI backend to deliver a complete, deployable AI-powered product — a directly applicable full-stack AI engineering skill.

05	Prompt Engineering	Practical proficiency in designing structured, context-rich prompts for large language models that produce consistent, high-quality outputs for specific use cases including recommendation and roadmap generation.
06	Database Design & SQLite Operations	Practical skills in designing relational schemas for student profiles, sessions, and conversation history using SQLite, with CRUD operations and analytical SQL queries.
07	PDF Report Generation	Ability to programmatically generate professional, data-populated PDF reports using ReportLab — applicable to business intelligence, EdTech, and enterprise reporting contexts.
08	Cloud Deployment on Render	Experience deploying Python-based full-stack applications to Render with environment variable management, production configurations, and live URL verification.

Dear Aryan,

Welcome to the NextGen Forge Technologies AI/ML Internship Program. This assignment has been thoughtfully designed to be fully achievable at **2–3 hours per day, Monday to Friday** — delivering genuine, portfolio-worthy Generative AI industry experience without overwhelming your schedule alongside your university commitments.

Over the next 3 months, you will design, build, and deploy the **AI Career Guidance & Admission Intelligence Platform** — a complete, production-ready EdTech application powered by the Gemini API, a RAG pipeline, and a professional FastAPI-Streamlit architecture. This is the type of project that demonstrates real-world AI engineering capability and directly differentiates candidates in the Generative AI, EdTech, and AI product development job market.

Approach each week with **curiosity** — explore what the Gemini API can do beyond the minimum task requirements; **discipline** — commit your 2–3 hours daily and honour weekly milestones; and **ownership** — treat every prompt, every RAG pipeline decision, and every Streamlit component as a reflection of your professional engineering identity.

Your Industry Guide **Aryan Sharma** and Reporting Manager **Sonia Sharma** are here to support your technical and professional growth throughout this journey. Raise blockers early, ask questions proactively, and never allow an unresolved issue to stall your progress beyond 24 hours — proactive communication is among the most valued professional behaviours in any AI engineering team.

The Generative AI and EdTech domains are among the fastest-growing and most impactful sectors in technology today. By completing this internship, you will carry with you not only technical skills but the confidence, portfolio evidence, and professional experience to pursue meaningful opportunities in AI product engineering, EdTech development, and intelligent systems design.

We are genuinely excited to be part of your journey. Welcome to the NextGen Forge family.

— The People & Engineering Team
NextGen Forge Technologies, Pune, Maharashtra

Issued By:

NextGen Forge Technologies

HR & People Operations

hr@nextgenforgetechnologies.com

Industry Guide:

Aryan Sharma

AI/ML Engineering Lead

Document Reference:

NFGT/HR/INT/2026/160

Date Issued: 19 June 2026